

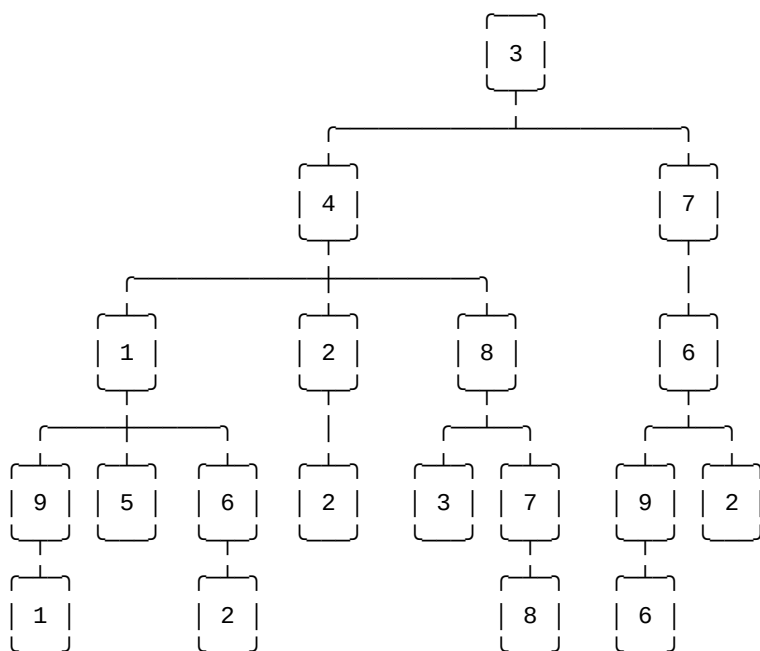
В тази задача ще работим с (кореново) дърво, в чиито върхове са съхранени числа. Всеки връх може да има произволен брой наследници. Редът на наследниците има значение. Първия наследник ще наричаме най-ляв, а последния – най-десен.

„Ниво с номер n “ наричаме списък от върховете в дървото, които са на разстояние n от корена, подредени от ляво надясно. Считаме, че коренът е на разстояние 0 от себе си.

Казваме, че ниво в дървото „представя“ даден вектор от цели числа, ако елементите във върховете на нивото, подредени от ляво надясно, образуват същата последователност, като тази съхранена във вектора. Например, ако векторът е {1, 2, 3}, той съвпада с последователност 1,2,3, но НЕ и с последователност 1,3,2.

Дърво от описания тип е показано по-долу. В него векторите, които съответстват на нивата, са както следва:

- * Ниво 0: {3}
- * Ниво 1: {4,7}
- * Ниво 2: {1,2,8,6}
- * Ниво 3: {9,5,6,2,3,7,9,2}
- * Ниво 4: {1,2,8,6}



Решете следните:

А) Да се дефинира тип наречен `TreeNode` за представяне на връх от такова дърво.

Б) Да се реализира функция:

```
int findLevel(const TreeNode* root, const int* numbers, int size)
```

която получава указател `root` към корен на дърво и връща номера на нивото представляващо масива `numbers` с размер `size`. Ако такова ниво няма, функцията връща -1. Ако има повече от едно ниво с такива елементи, да се върне най-близкото до корена. Ако масивът е празен, да се връща височината на дървото.

Например, за показаното по-горе дърво:

* Ако масивът е {3}, функцията трябва да върне 0.

* Ако масивът е {9,5,6,2}, функцията трябва да върне -1.

* Ако масивът е {1,2,8,6}, функцията трябва да върне 2.

* Ако масивът е празен, функцията трябва да върне 5.

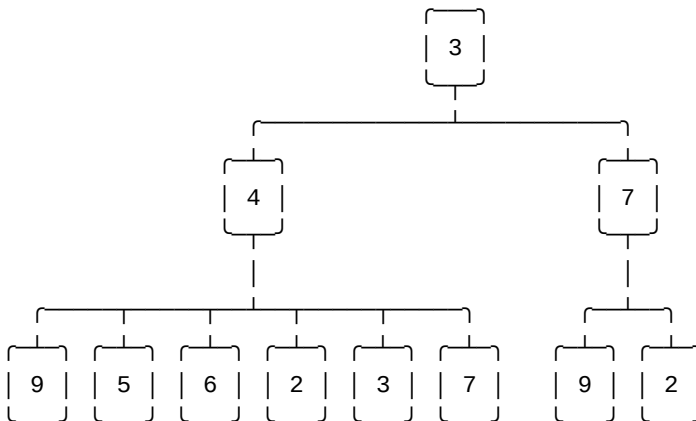
В) Да се реализира функция

`void removeAllLevels(TreeNode* root, const int* numbers, int size)`

която премахва всички нива с положителен номер от дървото с корен `root`, които представят масива `numbers`. Ако масивът е празен, функцията да не променя нищо.

При премахване на връх, неговите преки наследници заемат неговото място и стават преки наследници на родителя му.

Например, ако масивът е {1,2,8,6}, след изпълнението на функцията трябва да се получи следното дърво:



Г) Демонстрирайте използването на функциите в програма

Неориентиран граф е представен в текстов файл по следния начин:

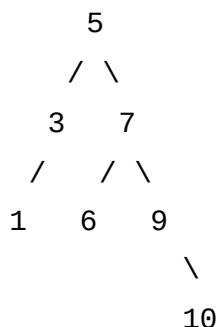
На първия ред на файла са записани две цели числа N и M - съответно броя на върховете и броя на ребрата в графа. Следват M реда, всеки съставен от три цели числа, описващи едно ребро. Първото и второто число определят върховете, които свързва съответното ребро, а третото число е дължина на реброто. Върховете са номерирани от 0 до $N-1$.

Напишете програма, която прочита граф от такъв файл, името на който се подава като аргумент от командния ред след това намерете и изведете на екрана броя на свързаните компоненти в графа. Също така изведете на екрана какъв е максималният брой ребра, които може да се премахнат, така че да се запази този брой свързани компоненти.

В тази задача е позволено използване на `std::vector`

Двоично дърво представяме като символен низ в следния формат:

- Празното дърво се представя със символа x
- Елемент на дървото се представя като:
(<число> (<ляво поддърво>) (<дясно поддърво>))
Например,
(5 (3 (1 x x) x) (7 (6 x x) (9 x (10 x x))))
е представянето на следното дърво



Напишете програма, която съдържа следните функции:

- Функция `deserialize`, която получава входен поток като аргумент. Тя трябва да прочита от него представяне на двоично дърво, да изгражда в паметта съответното дърво и да го връща като резултат. Ако данните не са коректни, да се хвърля подходящо изключение.
- Функция `serialize`, която получава като аргументи двоично дърво и изходен поток. Тя трябва да извежда представянето на дървото в потока.
- Функция `isWeightBalanced`, която получава като аргумент двоично дърво и проверява дали то е балансирано по тегло. Това означава, че за всеки възел в дървото броят на елементите в лявото му поддърво и броят на елементите в дясното му поддърво се различават най-много с едно.
- Функция `trimToBst`, която получава като аргумент двоично дърво и премахва от него всички елементи, които нарушават условието за двоично наредено дърво. Ако два елемента не са в правилна наредба един спрямо друг, да се премахва този, който е по-далече от корена. По този начин коренът никога не се премахва.
- Ако в някоя от функциите възникне грешка, да се хвърля подходящо изключение. Погрижете се да не допускате изтичане на памет.

Демонстрирайте поведението на функциите в кратка програма, която:

1. прочита от потребителя представяне на дърво;
2. конструира съответното му представяне в паметта;
3. проверява дали дървото е балансирано по тегло;
4. използва `trimToBst`, за да го сведе до двоично наредено дърво;
5. извежда представянето на полученото наредено дърво на екрана.

Неориентиран граф е представен в текстов файл по следния начин:

На първия ред на файла са записани две цели числа N и M - съответно броя на върховете и броя на ребрата в графа. Следват M реда, всеки съставен от три цели числа, описващи едно ребро. Първото и второто число определят върховете, които свързва съответното ребро, а третото число е дължина на реброто. Върховете са номерирани от 0 до $N-1$.

Дължините могат да бъдат отрицателни, но в графа няма отрицателни цикли.

Напишете програма, която прочита граф от такъв файл, името на който се подава като аргумент от командния ред, след това изисква от потребителя да въведе две числа - номер на начален връх и максимална дължина, след което трябва да изведе всички върхове, достижими от началния връх с път не по-дълъг от въведената максимална дължина.

В тази задача позволяваме да използвате `std::vector`